

Dive into Deep Learning (D2L)

LighTellite: Phase 3, Evaluation, and Discussion

Joon Soo Yoo, Ph.D.

Spring 2026

Assistant Professor

Defense & Advanced Security (DAS) Lab

Jeonbuk National University (JBNU)

Review Session

1. 왜 위성에서 이상 탐지인가?

- **역할:** 복잡한 시스템에서 라벨링되지 않은 예상치 못한 동작이나 악의적인 행위를 탐지함
- **원리:** 정상 운영 패턴을 학습하여, 이와 상충하는 실패(Failure)나 사이버 공격을 식별함
- **도메인 특성:** 위성, 특히 큐브위성(CubeSat)은 실패 시 막대한 손실이 발생하는 고신뢰성 시스템임

Challenge

큐브위성은 접근성(제작이 쉬움 - 부품 조달, 조립)이 높지만, 에너지와 연산 능력의 엄격한 한계로 인해 보안 위협에 더 취약함

2. 기존 딥러닝 방식의 한계

- **현황:** LSTM이나 Autoencoder(AE) 기반의 위성 전용 AD 모델들이 제안되어 왔음
- **치명적 약점:** 이러한 모델들은 높은 연산 부하와 그에 따른 **에너지 소모**를 수반함
- **현실적 문제:** 최고의 탐지 성능을 내는 고정 모델(Static Model)을 계속 돌리기엔 위성의 배터리가 버티지 못함

3. LighTellite의 핵심: 협력하는 이중 에이전트

- 동적 모델 선택(Dynamic Model Selection, DMS): 하나의 모델만 고집하지 않고, 상황에 맞춰 최적의 모델을 실시간 선택함
- Dual-Agent 메커니즘:
 - 에이전트 1 (Energy Manager): 사용 가능한 에너지 예산을 결정함
 - 에이전트 2 (Model Selector): 최신 데이터 분석을 통해 이상 탐지에 적합한 모델을 선별함

Proposed Goal

보안 성능과 에너지 효율 사이의 최적의 지점(Trade-off)을 찾아 자원 고갈 없는 지속 가능한 보안을 실현함

4. LighTellite의 주요 기여

- **최초의 에너지 인식형 AD**: 엄격한 에너지 제약 하에서 작동하는 위성용 AD 프레임워크 설계
- **RL 기반 DMS 적용**: 위성 도메인에 강화학습을 활용한 동적 모델 선택 기법을 최초로 도입함
- **정량적 우수성**: **AegisSat** 시뮬레이션 결과, 탐지율 10% 향상 및 에너지 21.8% 절감 달성
- **데이터 개방**: 58개 궤도의 큐브위성 시뮬레이션 데이터셋을 오픈소스로 공개함

Review - Methodology

Methodology Overview: LighTellite Pipeline

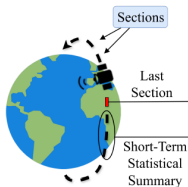
- **목표:** 온보드 데이터 패턴과 자원 상태에 따른 AD 성능 및 에너지 효율의 동적 균형
- **시스템 구조:** 총 3단계(Phase)의 순차적 파이프라인으로 구성됨

핵심 설계 철학

거시적 결정(Manager)과 미시적 실행(Worker)의 분리를 통한 효율적 자원 관리

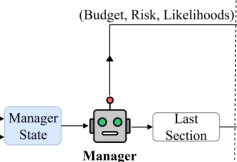
Methodology Overview: LighTellite Pipeline

Phase 1 - Section Preparation



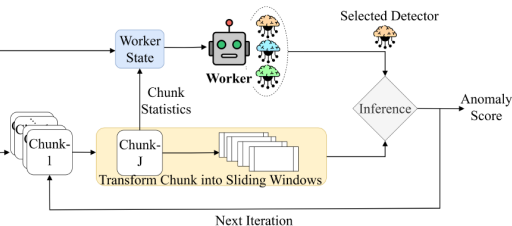
Satellite data is continuously collected and grouped into equal-duration sections. Last section's raw data, along with short-term statistical summary of recent sections is passed to the next phase.

Phase 2 - Budget Allocation



Manager uses short-term statistical summary to build its state to determine the energy budget, risk, and likelihood of each attack scenario. Last section and Manager outputs are passed to next phase.

Phase 3 - Model Selection and Anomaly Detection



The section is split into Chunks. For each Chunk, the Chunk's statistics, along with Manager outputs are used to build a state. Then, based on the state, the Worker selects an anomaly detection model from a pool of pretrained models and applies it to sliding windows to detect anomalies.

LighTellite Framework의 전체 워크플로우

Phase 1. Section Preparation: 데이터 그룹화

- **수집 단계:** 위성 궤도 운용 중 실시간 센서 데이터(Sensor Readings)를 지속적으로 읽어들이м
- **데이터 단위:** 수집된 데이터를 동일한 시간 길이의 '**섹션(Section)**' 단위로 그룹화함
- **추출 정보:**
 - Raw 데이터 (최근 섹션의 가공되지 않은 센서값)
 - 단기 통계 요약(Short-term Statistical Summary): 평균, 분산 등 통계적 특징

Input for Next Phase

이 섹션 정보와 통계 요약본이 Manager 에이전트의 판단 근거(State)가 됩니다.

Q. 위성은 대체 어떤 Raw 데이터를 수집하고 있나요?

- **질문 요약:** LighTellite가 처리하는 '섹션' 내 Raw 데이터에는 어떤 구체적인 센서값들이 포함되어 있나요?
- **답변:** 큐브위성의 상태를 나타내는 **텔레메트리(Telemetry)** 데이터로, 크게 3가지 카테고리로 나뉩니다.

AegisSat 기반 주요 수집 항목

카테고리	구체적 데이터 항목 (Raw Features)
전력 계통 (EPS)	버스 전압(Voltage), 배터리 전류(Current), 태양광 패널 발전량, 전력 소모량
컴퓨팅 자원 (OBC)	CPU 사용률(%), 메모리 점유율, 입출력(I/O) 작업 횟수, 프로세스 실행 시간
시스템 환경	내부/외부 온도, 통신 패킷 수(RF Traffic), 태양 가시성(일식 여부 플래그)

공학적 포인트

이 데이터들은 일정한 샘플링 주기에 따라 수집되어, 하나의 시점(t)에서 수십 개의 차원을 가진 **다변량 벡터(x_t)**를 구성하게 된단다.

Q. Raw 데이터가 이상 탐지에 미치는 영향

- **데이터의 결합:** 단일 센서값보다는 여러 값의 **상관관계**가 중요함
 - 예: CPU 사용량이 급증했는데 전압이 떨어진다? → 공격에 의한 자원 고갈 의심
 - 예: 일식 구간이 아닌데 발전량이 0이다? → 하드웨어 결함 의심
- **통계적 요약과 Raw Data의 역할:**
 - **Worker - Raw Data:** 미세한 떨림이나 즉각적인 이상 징후 포착
 - **Manager - Summary Data:** 평균, 분산 등을 통해 에너지 소모 트렌드나 시스템의 전반적인 건강 상태 파악

Note

결국 이 수많은 센서 데이터가 에이전트에게는 '눈'과 같다. 이 눈을 통해 현재 위성이 안전한지, 전기가 모자란지를 판단함.

Phase 2. Budget Allocation: 거시적 자원 배분

- 주체: Manager 에이전트 (지휘관 역할)
- 상태 구성(Manager State): 입력된 섹션 데이터를 바탕으로 현재 위성의 자원 상황을 파악함
- 결정 사항 (Manager Outputs):
 1. 에너지 예산(Energy Budget): 이번 섹션에서 사용할 총 에너지량 결정
 2. 위험 지수(Risk Factor): 현재 비정상 활동이 발생할 확률 계산
 3. 공격 시나리오 가능성: 각 공격 타입별 발생 확률 예측

Role of Manager

"지금은 위험하니 전기를 좀 더 써서 꼼꼼히 보자" 혹은 "평온하니 에너지를 아끼자"와 같은 전략적 판단을 내립니다.

Phase 3. Model Selection & AD: 미시적 탐지 실행

- 주체: **Worker 에이전트** (행동대장 역할)
- 세부 공정:
 1. 섹션을 더 작은 단위인 '**청크(Chunk)**'로 나눔
 2. **Contextual State 구축**: 전압/전류 변동, 에너지 트렌드, 최근 이상 이력을 결합
 3. **동적 모델 선택(Dynamic Model Selection, DMS)**: 사전 학습된 모델 풀(Pool)에서 최적의 모델 하나를 선택함
- **탐지(Inference)**: 선택된 모델을 슬라이딩 윈도우 방식으로 적용하여 **이상 점수(Anomaly Score)** 산출

The Core of DMS

각 모델은 성능과 에너지 소모량이 다르기 때문에, 자원 제약 내에서 가장 '가성비' 좋은 모델을 실시간으로 골라내는 것이 핵심입니다.

Phase 1 - Section Preparation

1. 궤도의 수학적 정의: Orbit and Sections

- 정의: 하나의 궤도(O_j)는 동일한 시간 길이를 가진 유한한 세그먼트인 '섹션(Section)'들의 시퀀스로 모델링됩니다.

수학적 표현 (Equation)

$$O_j = [S_{j,1}, S_{j,2}, \dots, S_{j,n}]$$

- O_j : j 번째 궤도 전체
 - $S_{j,i}$: j 번째 궤도 내의 i 번째 섹션 데이터
- 공학적 의미: 위성이 지구를 한 바퀴 도는 동안, LighTellite는 n 번의 의사결정 구간을 나누어 주기적으로 보안 상태를 점검함

2. 다변량 데이터의 구조화: Feature Groups

- 개념: 각 섹션 내의 수많은 센서 데이터를 성격에 따라 K 개의 그룹으로 분류합니다.
- 수학적 정의: $F = \{F_1, F_2, \dots, F_K\}$

Feature Grouping 사례

그룹 (F_k)	모니터링 대상	해당 센서 예시
$F_{telemetry}$	전력 시스템 (EPS)	전압, 전류, 발전량, 소모량
F_{system}	컴퓨팅 자원 (OBC)	CPU 점유율, 메모리, 네트워크 활동

Grouping의 목적

특정 리소스(예: 전력)의 이상 징후가 다른 리소스(예: 연산)에 미치는 영향을 독립적이면서도 유기적으로 분석하기 위함임

3. 입력 데이터 합성: Raw + Summary

- 매 활성화 주기마다 Manager 에이전트는 두 종류의 데이터를 입력받습니다.

1. 최신 Raw 데이터 ($S_{j,i}$):

- 현재 섹션에서 수집된 가장 따끈따끈한 원시 센서값
- 역할: 즉각적이고 미세한 변화(Spike) 탐지

2. 단기 통계 요약 (Statistical Summary):

- 현재 섹션 이전($S_{j,i-1}, S_{j,i-2}, \dots$)의 여러 섹션에서 유도된 통계치
- 역할: 시스템의 에너지 소모 추세 및 전반적인 건강 상태 파악

Next Step: Phase 2

이 합성된 정보를 기반으로 Manager는 "이번 섹션에서 전기를 얼마나 쓸 것인가?"를 결정하게 됩니다.

Phase 2 - Budget and Risk Assessment

1. Manager의 입력 정보: 상황 파악

- **Input:** 최신 섹션의 Raw 데이터($S_{j,i}$) + 이전 섹션들의 통계적 요약(Summary)
- **상태 벡터(s_i) 구성:** 위성의 운영 문맥(Operational Context)을 캡슐화함

State Vector (s_i)의 주요 요소

- **에너지 상태:** 정규화된 배터리 잔량, 에너지 소모 트렌드
- **임무 진행도:** 궤도 진행률(Orbit Progress)
- **시스템 건전성:** 섹션 단위의 텔레메트리 및 시스템 통계
- **보안 이력:** 최근 발생한 이상 탐지 기록

Note

지휘관은 단순히 현재 전기가 얼마 남았는지만 보는 게 아니라, "최근에 이상 징후가 나타난 적이 있는가?" 까지 고려해서 예산을 짭니다.

Q. 운영 문맥을 캡슐화한다는 게 구체적으로 무슨 뜻인가요?

- **질문 요약:** 최신 데이터와 과거 통계치를 합쳐서 '문맥'을 만든다는 것이 맞나요?
- **답변:** 네, 맞습니다. "현재 어디서 무엇을 하고 있는가"를 에이전트가 이해하도록 데이터를 조합하는 과정입니다.

입력 데이터와 운영 문맥의 매핑 (Mapping)

입력 소스 (Inputs)	생성되는 운영 문맥 (Operational Context)
최신 Raw ($S_{j,i}$)	현재 배터리 전압, CPU 점유율 등 ' 지금 이 순간 '의 상태
통계 요약 (Summary)	최근 1시간 동안의 에너지 소모 ' 트렌드 ' 및 평균 부하
궤도 정보 (i/n)	현재 위성이 태양광을 받는지(발전), 그림자인지(일식)의 ' 진행도 '
보안 이력	방금 전 청크에서 이상 점수가 높았는지에 대한 ' 과거 기억 '

Q. 왜 '단일 시점' 데이터만으로는 부족할까요?

- 사례: 전압이 3.2V로 떨어졌을 때
 - 문맥 없는 에이전트: "어? 전압이 낮네? 공격인가? (당황)"
 - 문맥을 아는 에이전트: "아, 지금은 일식 구간(궤도 진행도)이고, 최근 통계 요약을 보니 서서히 떨어지는 중이야. 이건 공격이 아니라 정상적인 소모야. 안심하자."

Note

데이터를 캡슐화(Encapsulation)한다는 것은, 흩어진 센서값들을 모아 에이전트가 즉각적으로 판단할 수 있는 하나의 '의미 있는 상태 벡터(s_i)'로 만드는 과정임.

2. 보상 함수의 재료: 무엇을 잘해야 상을 주는가?

- Manager의 정책 학습을 가이드하는 4가지 핵심 지표입니다.

Reward Components

지표	수식	공학적 의미
탐지 점수	$R_{det} = F + P$	Worker의 F1 스코어(F)와 임무 완수율(P)의 합
에너지 패널티	E_i	이번 섹션에서 Worker가 실제로 쓴 추론 에너지
예산 오차	B_{ex}	할당한 예산(B_i)과 실제 소모량 사이의 간극 (과잉 할당 방지)
고갈 패널티	D	배터리 방전 시 남은 궤도 비율 (조기 종료 방지)

Q. F1 스코어(F)와 임무 완수율(P)은 무엇인가요?

- **F1 스코어 (F):** 정밀도(Precision)와 재현율(Recall)의 조화 평균
 - 의미: "공격을 놓치지 않으면서도(Recall), 가짜 알람을 울리지 않는(Precision)" 균형 잡힌 성능 지표
- **임무 완수율 (P):** 이번 섹션에서 처리해야 할 전체 청크 중 실제로 처리한 청크의 비율
 - 수식: $P = \frac{\text{완료된 청크 수}}{\text{섹션 내 총 청크 수}}$
 - 사례: 에너지가 부족해 10개 청크 중 8개만 검사하고 멈췄다면 $P = 0.8$

Note

탐지 성능(F)이 아무리 좋아도, 전기가 모자라 검사를 끝까지 못 하면($P < 1$) 보안에 구멍이 뚫린 것이나 다름없다. 그래서 두 값을 합쳐서 평가하는 것임.

Q. 예산 오차(B_{ex})와 패널티(D)의 수치적 의미

- **예산 오차 (B_{ex}):** 관리자의 할당 예산(B_i)과 실제 소모량(E_i)의 차이
 - 수식: $B_{ex} = \max(0, B_i - E_i)$
 - 의도: 관리자가 에너지를 너무 '평평' 허락하지 않도록 조절함 (자원 최적화)
- **고갈 패널티 (D):** 배터리 방전 시 남은 궤도(섹션)의 비율
 - 예시: 총 10개 섹션 중 8번째에서 배터리가 다 나갔다면, 남은 2개 섹션에 대한 벌점
 $D = 2/10 = 0.2$
 - 의도: 에이전트가 단기 탐지 성능에 눈이 멀어 배터리를 일찍 다 써버리는 '조기 탈락'을 방지함

핵심 요약

B_{ex} 는 "낭비하지 마라", D 는 "끝까지 살아남아라"라는 교훈을 에이전트에게 주는 수단임.

3. 수학적 최적화: 통합 보상 함수 R_i

- Manager는 다음의 가중치 합을 최대화하도록 학습됩니다.

The Reward Equation

$$R_i = w_1 Y_i R_{det,i} - w_2 E_i - w_3 B_{ex,i} - (1 - Y_i) w_4 L_i - w_5 D_i$$

- $Y_i \in \{0, 1\}$: 현재 섹션에 공격이 존재하는지 여부
- $w_1, \dots, w_5$: 각 목표의 중요도를 조절하는 가중치(Weights)
- L_i : 정상 상황임에도 Manager가 공격 가능성을 높게 판단했을 때의 패널티

Key Logic

공격이 있을 때($Y_i = 1$)는 탐지 성능(R_{det})이 중요하고, 평시($Y_i = 0$)에는 불필요한 과잉 탐지(L_i)를 줄여야 한다!

4. Phase 2 Output: Worker에게 내리는 명령

- Manager가 모든 판단을 마치면 다음 정보를 다음 단계로 넘깁니다.
1. 할당된 예산 (B_i): 이번 섹션에서 쓸 수 있는 최대 에너지 주머니
 2. 위험 요인 (Risk Factor): 현재 환경의 위험 수준
 3. 공격 시나리오 가능성 (Likelihoods): 어떤 종류의 공격이 예상되는가?
 4. 섹션 데이터 ($S_{j,i}$): 실제 분석할 Raw 데이터

Transition to Phase 3

이제 행동대장(Worker)은 이 예산과 위험 정보를 들고, 실전에서 어떤 모델을 돌릴지 결정하게 됩니다.

4-1. 매니저의 Raw 데이터 + 단기 요약 데이터에서 결정으로: Mapping Process

- Manager 내부에는 데이터를 판단 결과로 변환하는 **정책 신경망(π_θ)**이 존재합니다.

1. 상태 구성 (State Construction):

- 섹션 Raw($S_{j,i}$)와 이전 통계를 하나의 긴 벡터(s_i)로 결합(Concatenation)함
- 이때 모든 값은 신경망이 잘 이해하도록 0 ~ 1 사이로 **정규화(Normalization)**됨

2. 신경망 통과 (Neural Inference):

- 결합된 벡터 s_i 가 심층 신경망(DNN)의 입력층으로 들어감
- 다중 레이어를 거치며 데이터 간의 복잡한 상관관계를 추론함

4-2. 출력값의 결정: Budget, Risk, Likelihoods

- 신경망의 마지막 층(Output Layer)은 우리가 원하는 3가지 값을 내놓습니다.

Manager의 출력값(Action) 도출

출력 항목	활성화 함수	도출 의미
예산 (B_i)	Sigmoid/Linear	이번 섹션에서 허용할 에너지의 총량 (Joule 단위)
위험도 (Risk)	Sigmoid (0 ~ 1)	현재 환경이 비정상(Anomaly)일 확률
공격 가능성	Softmax	DoS, 자원 고갈 등 각 시나리오가 발생할 상대적 확률

How does it know?

이 신경망은 지난 슬라이드에서 배운 **보상 함수(R_i)**를 최대화하는 방향으로 수만 번의 시행착오를 거치며 '예산 짜는 법'을 스스로 터득한 것입니다.

4-3. Manager의 신경망 업데이트: 전략적 진화

- **주체:** Manager Policy Network (π_{θ_M})
- **학습 목표:** 보상 함수 R_i 의 기댓값 최대화

업데이트 로직 (The Learning Loop)

1. **Action:** 예산(B_i)과 위험도를 결정함
2. **Feedback:** Worker가 실제 수행한 결과(탐지 점수, 에너지 소모)를 관찰함
3. **Reward:** R_i 를 계산(일종의 '칭찬' 혹은 '벌' 개념)
4. **Update:** 보상이 낮으면 신경망 가중치(θ_M)를 수정하여 다음번에 더 나은 예산을 짜도록 함

4-Final. Worker의 신경망 업데이트: 모델 선택의 묘미

- 주체: Worker Policy Network (π_{θ_w})
- 학습 목표: Manager가 준 예산(B_i) 내에서 탐지 성능 최대화

Worker의 Action: Dynamic Model Selection

상태(State)	행동(Action: 모델 선택)
예산이 넉넉하고 위험함	Heavy VAE: 에너지를 써서라도 확실히 탐지
예산이 적고 평온함	Light VAE: 에너지를 아끼며 최소한의 감시 유지

공학적 결론

두 에이전트는 서로 독립된 신경망을 가지지만, '예산'이라는 고리로 묶여 함께 성장합니다. 이걸 **계층적 강화학습(HRL)** 구조라고 부름.

C. Phase 3

Model Selection and Anomaly Detection

1. Phase 3의 역할: Worker가 실제 탐지를 수행

- **입력:** Manager가 넘긴 섹션 데이터 $S_{j,i}$, 에너지 예산 B_i , 위험도, 공격 시나리오 likelihood
- **주체:** Worker agent
- **목표:** 주어진 에너지 예산 안에서 이상 탐지 성능을 최대화
- **핵심 기법:** Dynamic Model Selection(DMS)

Teaching Point

Manager는 “이번 섹션에 얼마를 쓸지” 결정하고, Worker는 “그 예산 안에서 어떤 탐지 모델을 언제 쓸지” 결정합니다.

2. Stage 1: Section을 Chunk로 나누기

- Worker는 섹션 $S_{j,i}$ 를 더 작은 m 개의 chunk로 분할합니다.

Section-to-Chunk

$$S_{j,i} = [C_1, C_2, \dots, C_m]$$

- 각 chunk는 짧은 고정 시간 구간의 telemetry/system 데이터를 포함합니다.
- 이렇게 나누면 섹션 전체에 하나의 모델을 고정하지 않고, 시간에 따라 탐지 전략을 바꿀 수 있습니다.

왜 더 작게 나누는가?

공격 징후와 에너지 상태는 섹션 내부에서도 변할 수 있으므로, chunk 단위 의사결정이 더 민감한 적응을 가능하게 합니다.

3. Stage 1-2: Chunk를 Sliding Window로 변환

- 각 chunk C_t 는 AD 모델이 처리할 수 있는 sliding window들의 시퀀스로 바뀝니다.

Chunk-to-Window

$$C_t = [W_{t,1}, W_{t,2}, \dots, W_{t,y}]$$

- VAE 기반 AD 모델은 window 단위로 정상 패턴을 재구성합니다.
- 재구성 오차가 threshold보다 크면 anomalous window로 판단합니다.
- 즉, 실제 이상 여부 판단은 chunk 안의 window들에서 발생합니다.

Example: Sliding Window

- Sliding window는 연속된 시계열 데이터를 작은 구간으로 나누는 방법입니다.
- 하나의 값이 아니라 **짧은 시간 패턴**을 보기 위해 사용합니다.

Example (Window Size is 3)

Time:	t1	t2	t3	t4	t5	t6
Value:	3.3	3.2	3.3	3.4	2.9	2.8

W1 = [3.3, 3.2, 3.3]

W2 = [3.2, 3.3, 3.4]

W3 = [3.3, 3.4, 2.9]

W4 = [3.4, 2.9, 2.8]

- 각 window는 서로 겹치며 이동합니다 (sliding)
- 단일 값이 아닌 **패턴 변화**를 탐지할 수 있습니다.

Key Idea

이상 탐지는 "값"이 아니라 **시간에 따른 변화 패턴**을 보는 문제입니다.

Example: Variational Autoencoder (VAE)

- VAE는 입력 데이터를 압축했다가 다시 복원(reconstruction)하는 모델입니다.

Basic Structure

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow x'$$

- x : 입력 데이터 (window)
- x' : 복원된 데이터
- 목표: $x \approx x'$

Anomaly Detection

$$\text{Reconstruction Error} = |x - x'|$$

- 정상 데이터 $\rightarrow$ 잘 복원됨 $\rightarrow$ error 작음
- 이상 데이터 $\rightarrow$ 복원 실패 $\rightarrow$ error 큼

Key Idea

VAE는 정상 패턴만 학습하기 때문에,
이상 데이터는 잘 복원하지 못한다는 성질을 이용합니다.

4. Stage 2: Worker State 구성

- Worker는 chunk마다 현재 의사결정에 필요한 상태 s_t 를 구성합니다.
- 이 상태는 “지금 어떤 모델을 돌릴지”를 판단하기 위한 요약 벡터입니다.

Worker State에 포함되는 정보

정보 종류	의미
남은 에너지 예산	현재 chunk 이후에도 탐지를 계속할 수 있는지 판단
Telemetry 추세	전압, 전류, 배터리 관련 변화 포착
System 상태	CPU, 메모리, 네트워크 I/O 등 시스템 활동 포착
Manager 출력	risk factor와 공격 시나리오 likelihood 반영
최근 탐지 이력	직전 chunk들의 anomaly pattern과 선택 모델 기록

5. Stage 3: 후보 모델 Pool과 Action

- 각 feature group $f \in F$ 마다 두 종류의 사전 학습된 detector가 있습니다.

AD Model Pool

모델	장점	비용
Lightweight L_f	에너지 소모가 낮음	탐지 성능이 상대적으로 낮을 수 있음
Heavy H_f	탐지 정확도가 높음	더 많은 연산과 에너지 필요

Action Set

$$D = \{L_f, H_f \mid f \in F\}$$

6. Stage 3: 모델 선택과 예산 업데이트

1. Worker가 현재 상태 s_t 를 관찰합니다.
2. 후보 모델 중 하나를 선택합니다.
3. 선택한 모델을 chunk의 sliding windows에 적용해 anomaly score를 계산합니다.
4. 실제 사용한 에너지 $E_{used,t}$ 를 섹션 예산에서 차감합니다.

Budget Update

$$B_i^{(t+1)} = B_i^{(t)} - E_{used,t}$$

Safety Threshold

남은 예산이 안전 임계값보다 낮아지면 Worker는 해당 섹션의 추가 처리를 중단합니다.

6-1. Candidate Models (모델 선택 공간)

- Worker는 여러 개의 사전 학습된 AD 모델 중 하나를 선택합니다.
- 모델은 feature group과 복잡도에 따라 나뉩니다.

Model Pool

- Telemetry: L_{tele} (Light), H_{tele} (Heavy)
- System: L_{sys} (Light), H_{sys} (Heavy)

- 총 4개의 후보 모델이 존재합니다.
- Worker는 매 chunk마다 하나의 모델을 선택합니다.

Key Idea

하나의 모델을 고정하지 않고,
상황에 따라 다른 모델을 선택 (Dynamic Model Selection)

6-2. VAE는 사전 학습된 모델

- VAE 기반 AD 모델은 미리 학습되어 있습니다.
- 학습은 정상 데이터만을 사용하여 수행됩니다.

Offline Training

- 정상 데이터로 VAE 학습
 - 모델 파라미터 고정
-
- 실제 위성 운영 중에는 학습이 아니라 **추론(inference)**만 수행
 - Worker는 모델을 학습하지 않고 선택만 합니다.

Key Idea

VAE = "이미 학습된 탐지기"

Worker = "어떤 탐지기를 쓸지 결정하는 컨트롤러"

6-3. Anomaly Score 계산 과정

- 각 chunk는 여러 개의 sliding window로 구성됩니다.
- 각 window는 VAE에 입력됩니다.

Reconstruction

$$x \rightarrow \text{VAE} \rightarrow x'$$

Anomaly Score

$$\text{Error} = |x - x'|$$

- 정상 데이터 → reconstruction error 작음
- 이상 데이터 → reconstruction error 큼
- threshold를 기준으로 anomaly 여부 결정

Key Idea

VAE는 정상 패턴만 학습하므로,
이상 데이터는 복원하지 못한다

6-4. Energy Budget Update

- Worker는 각 chunk에서 모델을 실행할 때 에너지를 소비합니다.
- 사용한 에너지는 섹션 예산에서 차감됩니다.

Budget Update

$$B_i^{(t+1)} = B_i^{(t)} - E_{used,t}$$

- $E_{used,t}$: 현재 chunk에서 사용된 에너지
- Heavy 모델 → 에너지 많이 사용
- Lightweight 모델 → 에너지 적게 사용

Key Idea

모델 선택은 단순 정확도가 아니라
에너지 소비와의 trade-off 문제

6-5. 전체 흐름: Chunk 기반 반복

- Worker는 각 chunk마다 다음 과정을 반복합니다:

Processing Loop

1. 현재 상태 `s_t` 관찰
2. 모델 선택
3. VAE inference 수행
4. anomaly score 계산
5. 에너지 차감

- 이 과정은 모든 chunk에 대해 반복됩니다.
- 예산이 부족하면 탐지가 중단됩니다.

Stopping Condition

남은 예산이 임계값 이하가 되면
탐지 중단 (Safety Mechanism)

7-1. Worker Reward: 무엇을 배우는가?

- Worker는 각 chunk마다 reward를 받아 **모델 선택 전략**을 학습합니다.
- 목표: 탐지 성능을 유지하면서 **에너지 소비 최소화**

핵심 질문

- 어떤 모델을 선택해야 하는가?
- 언제 Heavy 모델을 써야 하는가?
- 언제 Lightweight 모델로 충분한가?

7-2. Per-Chunk Reward Function

Reward Function

$$r_t = \lambda_A \text{Align}_t + \lambda_H Y_t H_t - \beta_E E_t - \beta_B \mathbb{I}[E_t > B_i]$$

- 각 항은 "좋은 선택"과 "나쁜 선택"을 구분하기 위한 신호입니다.
- Worker는 이 reward를 최대화하도록 policy를 학습합니다.

Key Idea

정확도 + 에너지 + 예산 제약을 동시에 고려하는 reward

7-3. Reward 변수 설명 (1)

- $Align_t \in \{0, 1\}$
 - 선택한 모델의 feature group이 현재 상황과 잘 맞는가?
 - 예: CPU 공격 → System model 선택 시 1
- $Y_t \in \{0, 1\}$
 - 현재 chunk에 공격이 존재하는가?
- $H_t \in \{0, 1\}$
 - Heavy 모델을 선택했는가?
 - 1: Heavy (정확도 ↑, 에너지 ↑)
 - 0: Light (정확도 ↓, 에너지 ↓)
- E_t
 - 현재 chunk에서 사용된 에너지
- $\mathbb{I}[E_t > B_i]$
 - 예산 초과 여부
 - 초과 시 1, 아니면 0
- $\lambda_A, \lambda_H, \beta_E, \beta_B$ 는 학습되는 값이 아닙니다
- 사람이 설정하는 하이퍼파라미터입니다

역할

- λ_A : alignment 중요도
- λ_H : heavy 모델 보상 정도
- β_E : 에너지 패널티 크기
- β_B : 예산 초과 패널티 크기

8-1. 무엇이 학습되는가?

- 이 시스템에는 두 개의 RL agent가 존재합니다:
 - Manager: 에너지 예산 결정
 - Worker: 모델 선택

핵심 질문

Reward 함수의 파라미터가 학습되는가?

정답

- 학습되는 것: Policy Network (신경망 파라미터)
- 학습되지 않는 것: Reward weight (w_i, λ, β)

Key Insight

Reward는 학습 기준이고,
Policy는 학습 대상입니다.

8-2. Reward Weight는 무엇인가?

- Manager reward:

$$R_i = w_1 Y_i R_{det,i} - w_2 E_i - \dots$$

- Worker reward:

$$r_t = \lambda_A Align_t + \lambda_H Y_t H_t - \beta_E E_t - \dots$$

Weight Parameter

- $w_1, \dots, w_5$
- λ_A, λ_H
- β_E, β_B

- 모두 하이퍼파라미터 (사람이 설정)
- 학습 과정에서 업데이트되지 않음

왜 고정인가?

Reward까지 학습하면 objective가 불안정해지고
학습 방향이 모호해질 수 있음

8-3. 전체 학습 과정 (Offline Training)

Step-by-Step Training

1. 데이터 수집 (Telemetry, System)
2. VAE 모델 학습 (정상 데이터)
3. Worker RL 학습
4. Manager RL 학습 (Worker freeze)
5. Joint fine-tuning

- 모든 학습은 **오프라인 환경에서 수행**
- 실제 위성에서는 학습이 수행되지 않음

Key Insight

이 시스템은 **offline-trained policy + online inference** 구조

8-4. Onboard 동작 (Inference Only)

Runtime Flow

Manager → 에너지 예산 결정

Worker → 모델 선택

VAE → anomaly detection 수행

- 위성에서는 학습이 아니라 **결정(inference)**만 수행
- 이미 학습된 정책을 사용하여 실시간 의사결정

중요한 특징

- 계산량 최소화
- 안정적인 동작
- 실시간 대응 가능

Key Insight

위성은 "학습하는 시스템"이 아니라
학습된 정책을 실행하는 시스템

9-1. Policy Network란 무엇인가?

- Policy Network는 현재 상태(state)를 입력으로 받아 행동(action)을 결정하는 신경망입니다.

Policy

$$\pi(a | s)$$

- 입력: 현재 상태 s
- 출력: 행동 a 또는 행동 확률

이 논문에서의 역할

- Manager: 에너지 예산 결정
- Worker: 모델 선택 결정

9-2. Manager vs Worker Policy Network

- 두 개의 Policy Network가 존재합니다:

Worker Policy

입력: chunk 상태 s_t

출력: 선택할 모델

→ $\{L_{tele}, H_{tele}, L_{sys}, H_{sys}\}$

Manager Policy

입력: section 상태

출력:

- energy budget
- risk factor
- attack likelihood

Key Insight

Worker는 "어떤 모델을 쓸지",
Manager는 "얼마를 쓸지" 결정합니다.

9-3. Neural Network 구조

- Manager와 Worker 모두 동일한 구조의 신경망을 사용합니다.

Network Architecture

Input (state vector)

↓

Fully Connected (64)

↓

Fully Connected (64)

↓

Output (action)

- 은닉층 2개
- 각 층 64개의 뉴런

핵심

이 구조는 논문에서 설정된 하이퍼파라미터입니다.

9-4. 네트워크 구조는 누가 정하는가?

- 신경망 구조는 학습되는 것이 아닙니다.

Hyperparameters

- 레이어 개수
- 뉴런 개수 (64, 64)
- 활성화 함수
- PPO 설정

- 모두 사람이 설계하는 요소입니다.

Key Insight

구조는 설계되고, weight만 학습됩니다.

9-5. 왜 FCNN을 사용하는가?

- 이 논문에서 사용하는 신경망은 **FCNN (Fully Connected Neural Network)**입니다.
- 입력 데이터:
 - telemetry 통계
 - system 상태
 - 에너지 정보

왜 FCNN인가?

- 입력이 이미지가 아님 → CNN 불필요
- 시퀀스 모델링이 직접적이지 않음 → RNN 불필요
- 단순한 상태 벡터 → FCNN이 가장 적절

9-6. 무엇이 학습되는가?

- 학습되는 것은 Policy Network의 weight입니다.

학습되는 것

- 신경망 가중치 (weights)

학습되지 않는 것

- 네트워크 구조
- reward weight (w_i, λ, β)

핵심

Reward는 기준이고, Policy는 학습 대상입니다.

V. Evaluation

Dataset, Experimental Settings, and Results

1. Evaluation의 세 가지 질문

Research Questions

RQ	질문
RQ1	Feature-specific detector가 combined feature detector보다 나은가?
RQ2	AD 모델의 복잡도와 에너지 소모는 어떻게 다른가?
RQ3	LighTellite는 static AD model보다 탐지와 에너지 측면에서 나은가?

- 평가의 핵심 비교 대상은 **static model**입니다.
- Static model은 궤도 전체에서 같은 AD 모델만 계속 사용합니다.

2. Data Collection: AegisSat Testbed

- 테스트베드: AegisSat, CubeSat 사이버보안 실험 환경
- 데이터 규모: 58개 full-orbit simulation
- 궤도 길이: 각 orbit은 약 90분, 약 5,000 samples
- 정상/공격 분포: 23개 benign orbit, 35개 attack 포함 orbit
- 전체 샘플: 230K 이상

수집 데이터 종류

Feature Group	예시
Telemetry	EPS 전압/전류, 배터리 전압/전류, 배터리 온도
System	CPU utilization, context switches, interrupts, network I/O

3. Attack Scenarios

실험 공격 시나리오

공격	동작	주요 영향
Excessive Resource Usage	파일 생성/삭제, 메모리 할당, CPU loop를 병렬 실행	CPU, memory, I/O, 전력 소모 증가
Data Leakage via RF	저장된 이미지를 RF transmitter로 외부 수신자에게 전송	네트워크 I/O 및 통신 활동 이상
Hybrid	두 공격이 함께 나타남	telemetry와 system 모두에 복합 징후 발생

Labeling

공격은 무작위 orbit의 무작위 시간 구간에서 활성화되며, 해당 activation period가 평가용 label로 사용됩니다.

4. AD Models and RL Agents

- **AD 모델:** VAE 기반 anomaly detector
- **학습 방식:** benign orbit만 사용한 unsupervised training
- **모델 구성:** telemetry, system, combined feature group 각각에 lightweight/heavy 모델 구성
- **Threshold:** validation set 기반 threshold를 넘는 score를 anomaly로 판단

RL Agent 설정

Agent	Algorithm	Action
Worker	PPO	feature group별 lightweight/heavy AD 모델 선택
Manager	PPO	energy budget, attack likelihoods, risk factor 결정

D. Results

What did LighTellite learn?

1. Table I: Static AD 모델의 탐지 성능

Attack Scenario	Features	Model	Precision	Recall	F1
Resource Usage	Telemetry	Lightweight	0.88	0.81	0.83
		Heavy	0.90	0.91	0.91
	System	Lightweight	0.75	1.00	0.85
		Heavy	0.76	1.00	0.86
	Combined	Lightweight	0.91	0.71	0.79
		Heavy	0.83	0.78	0.80
Data Leakage	Telemetry	Lightweight	0.22	0.06	0.08
		Heavy	0.25	0.07	0.10
	System	Lightweight	0.72	0.99	0.83
		Heavy	0.95	0.99	0.97
	Combined	Lightweight	0.82	0.13	0.21
		Heavy	0.59	0.75	0.64
Hybrid	Telemetry	Lightweight	0.71	0.59	0.63
		Heavy	0.79	0.67	0.71
	System	Lightweight	0.36	1.00	0.52
		Heavy	0.37	1.00	0.53
	Combined	Lightweight	0.89	0.58	0.69
		Heavy	0.74	0.76	0.75

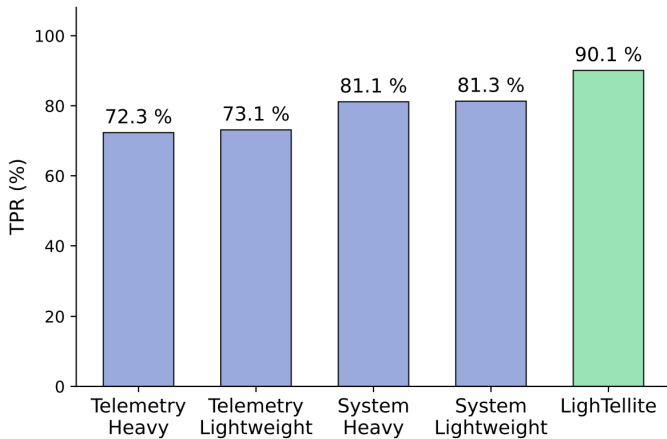
2. Table 1 해석: Feature Specialization

- **Resource Usage:** telemetry heavy가 F1 0.91로 가장 높습니다.
- **Data Leakage:** system heavy가 F1 0.97로 가장 높습니다.
- **Hybrid:** combined 모델이 항상 최선은 아니며, domain-specific 모델도 성능 저하를 보입니다.

핵심 메시지

공격 종류마다 잘 드러나는 feature가 다릅니다. 따라서 하나의 고정 모델보다, 상황에 맞게 모델을 고르는 DMS가 유리해질 수 있습니다.

3. Fig. 2: TPR 비교



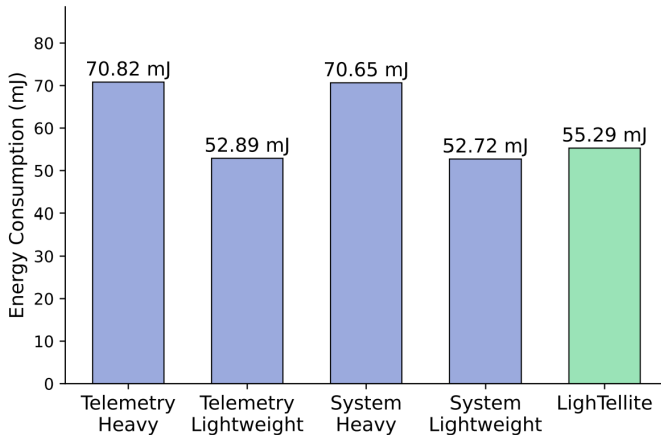
4. Fig. 2 해석: 공격 탐지율

- TPR(True Positive Rate)은 공격 window 중 실제로 공격이라고 탐지한 비율입니다.
- Static AD 모델들은 72.3%–81.3% 범위의 TPR을 보입니다.
- LighTellite는 **90.1% TPR**을 달성합니다.

Result

동적 모델 선택을 통해 static model 대비 공격 탐지율을 약 10%p 수준으로 끌어올렸습니다.

5. Fig. 3: Orbit 전체 에너지 소모 비교



6. Fig. 3 해석: 에너지 효율

- Heavy 모델은 orbit당 약 71 mJ를 소비합니다.
- Lightweight 모델은 orbit당 약 53 mJ를 소비합니다.
- LighTellite는 평균 **55.29 mJ**를 소비합니다.
- 이는 heavy 모델 대비 최대 **21.8% 낮은 에너지 소모**입니다.

좋은 trade-off

LighTellite는 lightweight lower bound에 가깝게 에너지를 쓰면서, 탐지 성능은 가장 높게 유지합니다.

7. Table II: 모델 복잡도와 에너지 분포

Features	Model	Parameters	MFLOPS	Energy Distribution (mJ)	
				Mean	Std.
Telemetry	Lightweight	14,519	1.40	6.61	0.07
	Heavy	37,715	3.77	8.85	0.06
System	Lightweight	14,348	1.38	6.59	0.07
	Heavy	37,448	3.73	8.83	0.06

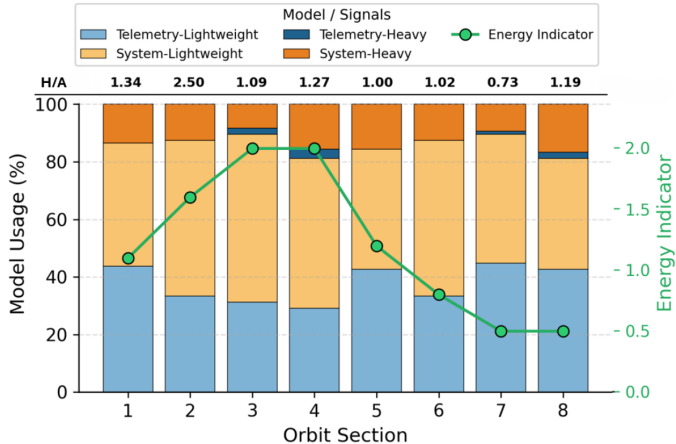
8. Table II 해석: Heavy는 왜 비싼가?

- Heavy 모델은 lightweight 모델보다 parameter와 MFLOPs가 약 3배 많습니다.
- Lightweight 모델은 inference당 약 6.6 mJ, heavy 모델은 약 8.8 mJ를 소비합니다.
- Telemetry와 system 모델의 에너지 패턴은 거의 비슷합니다.

Interpretation

에너지 차이는 input feature 자체보다 모델 architecture의 크기에서 주로 발생합니다.

9. Fig. 4: Worker의 모델 선택 패턴



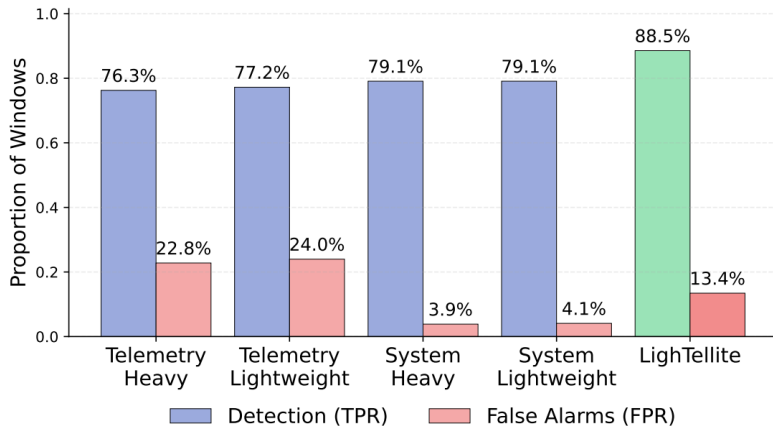
10. Fig. 4 해석: 언제 Heavy를 쓰는가?

- 막대는 섹션별 Worker의 AD 모델 선택 빈도를 나타냅니다.
- 초록색 선은 Energy Indicator로, CubeSat의 에너지 상태를 나타냅니다.
- H/A ratio는 anomaly 비율 대비 heavy model 사용 비율입니다.
- 에너지 상태가 좋은 Sections 1–4에서는 H/A ratio가 1.09–2.50으로 높습니다.
- 에너지 제약이 큰 Sections 5–8에서는 heavy 사용을 줄여 H/A ratio가 0.73–1.19로 낮아집니다.

정책의 의미

Worker는 공격 가능성이 높고 에너지가 충분할 때 heavy model을 쓰고, 에너지가 부족하면 공격 징후가 있어도 보수적으로 움직입니다.

11. Fig. 5: TPR과 FPR 동시 비교



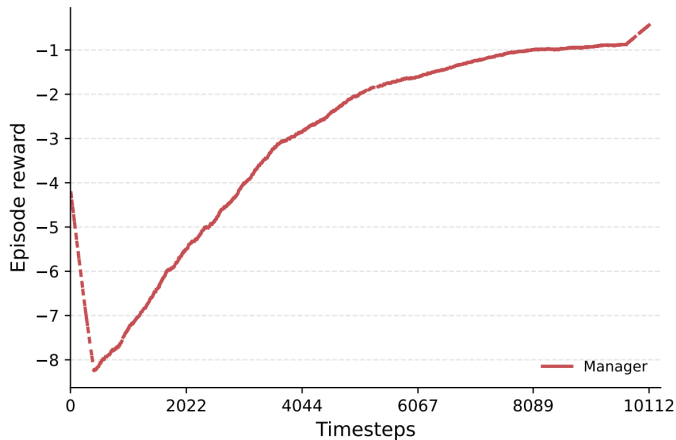
12. Fig. 5 해석: 공격을 잘 잡고, 오탐은 낮게

- Fig. 5는 공격 탐지율(TPR)과 정상 window를 공격으로 잘못 판단한 비율(FPR)을 함께 보여줍니다.
- LighTellite는 가장 높은 TPR을 달성합니다.
- 동시에 FPR을 지나치게 높이지 않아, 실제 운용에서 alert fatigue를 줄일 수 있습니다.

수업 포인트

보안 모델은 탐지만 높으면 끝이 아닙니다. 위성 운용에서는 오탐도 에너지와 운영 부담으로 이어질 수 있습니다.

13. Fig. 6: Manager Reward 학습 곡선



14. Fig. 6 해석: 정책 최적화

- Fig. 6은 Manager agent의 smoothed mean episode reward 변화를 보여줍니다.
- Manager reward는 Worker의 탐지 성능과 에너지 사용량을 포함합니다.
- 따라서 이 곡선은 두 agent의 협력 정책이 학습되었는지를 간접적으로 보여줍니다.

중요한 해석

reward가 안정적으로 개선된다는 것은 Manager가 예산을 더 잘 배분하고, Worker가 그 안에서 더 적절한 모델을 선택하도록 함께 적응했다는 뜻입니다.

VI. Discussion

Limitations and Broader Implications

1. Limitations

논문에서 제시한 한계

한계	설명
External operating factors	방사선, 온도 변화, 하드웨어 노화가 센서값과 모델 안정성에 영향을 줄 수 있음
Model pool diversity	사전 학습된 AD 모델 pool의 품질과 다양성에 성능이 의존함
Real mission transition	AegisSat simulation이 실제 장기 궤도 환경을 완전히 대체하지는 못함

Mitigation

실제 임무에서는 in-orbit recalibration, 실제 CubeSat 데이터 추가, 더 다양한 detector pool이 필요합니다.

2. Broader Implications

- **Energy-aware AD:** 에너지 효율을 부가 목표가 아니라 탐지 시스템의 핵심 설계 원리로 사용
- **RL-based DMS:** 제한된 자원 환경에서 RL이 context-aware decision-making 도구가 될 수 있음을 보임
- **Beyond CubeSats:** IoT, autonomous vehicles처럼 에너지와 연산 자원이 제한된 도메인으로 확장 가능

Takeaway

이 논문의 가치는 “위성에서 AD를 했다” 만이 아니라, 자원 제약이 심한 시스템에서 AI를 어떻게 선택적으로 실행할 것인가를 보여준 데 있습니다.

VII. Conclusion

and Future Work

1. Conclusion

- LighTellite는 Manager-Worker 구조의 RL 기반 AD framework입니다.
- Manager는 섹션 단위 예산과 위험도를 결정합니다.
- Worker는 chunk 단위로 AD 모델을 동적으로 선택합니다.
- 실험 결과, static AD 모델보다 더 좋은 탐지-에너지 trade-off를 달성했습니다.

핵심 결론

Contextual awareness가 efficiency를 해치지 않고 detection performance를 높이는 핵심 요인입니다.

2. Future Work

- 더 다양한 attack scenario 추가
- 새로운 detector architecture와 feature set 확장
- 실제 orbiting CubeSat 데이터 기반 검증
- 장기적인 하드웨어 노화, 방사선, 온도 변화에 대한 robustness 평가

Any Question?